

APPENDIX

This appendix contains the supporting source code listings and running-system snapshots for the Shifty project. The code modules are organised by subsystem and the snapshots illustrate the end-user experience across the principal surfaces of the application.

Appendix A: USER SCHEMA AND PASSWORD HASHING

```
// models/User.js — Mongoose schema for the User collection
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const AddressSchema = new mongoose.Schema({
  label: { type: String, required: true },    // "Home", "Office"
  line1: { type: String, required: true },
  city: { type: String, required: true },
  pincode: { type: String, match: /^\\d{6}$ / },
  lat: Number,
  lng: Number
}, { _id: false });

const UserSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  phone: { type: String, required: true, match: /^\\d{10}$ / },
  password_hash: { type: String, required: true },
  saved_addresses: [AddressSchema],
  role: { type: String, enum: ['customer', 'mover', 'admin'], default: 'customer' },
  created_at: { type: Date, default: Date.now }
});

// Hash password before saving
UserSchema.pre('save', async function (next) {
  if (!this.isModified('password_hash')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password_hash = await bcrypt.hash(this.password_hash, salt);
  next();
});
```

```
// Helper to verify password on login
UserSchema.methods.comparePassword = async function (plain) {
  return bcrypt.compare(plain, this.password_hash);
};

module.exports = mongoose.model('User', UserSchema);
```

Appendix B: AUTHENTICATION CONTROLLERS

```
// controllers/authController.js — Sign-up and login endpoints
const User = require('../models/User');
const jwt = require('jsonwebtoken');

const JWT_SECRET = process.env.JWT_SECRET;
const TOKEN_TTL = '7d';

function signToken(user) {
  return jwt.sign(
    { sub: user._id, role: user.role },
    JWT_SECRET,
    { expiresIn: TOKEN_TTL }
  );
}

// POST /api/auth/signup
exports.signup = async (req, res) => {
  try {
    const { name, email, phone, password } = req.body;
    if (!email || !password || password.length < 8) {
      return res.status(400).json({ message: 'Weak or missing credentials' });
    }
    const existing = await User.findOne({ email });
    if (existing) return res.status(409).json({ message: 'Email already registered' });

    const user = await User.create({
      name, email, phone,
      password_hash: password // Hashed in Schema pre-save hook
    });
  }
}
```

```

    return res.status(201).json({
      token: signToken(user),
      user: { id: user._id, name: user.name, email: user.email }
    });
  } catch (err) {
    console.error('Signup error:', err);
    return res.status(500).json({ message: 'Internal Server Error' });
  }
};

// POST /api/auth/login
exports.login = async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user || !(await user.comparePassword(password))) {
    return res.status(401).json({ message: 'Invalid email or password' });
  }
  return res.status(200).json({
    token: signToken(user),
    user: { id: user._id, name: user.name, email: user.email, role: user.role }
  });
};

```

Appendix C: TOKEN VERIFYING MIDDLEWARE

```

// middleware/authMiddleware.js — JWT verification for protected routes
const jwt = require('jsonwebtoken');

module.exports = function requireAuth(req, res, next) {
  const header = req.headers.authorization || '';
  if (!header.startsWith('Bearer ')) {
    return res.status(401).json({ message: 'Missing bearer token' });
  }
  const token = header.slice('Bearer '.length);
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    req.auth = { userId: payload.sub, role: payload.role };
    return next();
  } catch (err) {

```

```

    return res.status(401).json({ message: 'Invalid or expired token' });
  }
};

// Role-based guard - e.g. router.patch('/mover/status', requireAuth, requireRole('mover'),
handler)
module.exports.requireRole = function (role) {
  return (req, res, next) => {
    if (!req.auth || req.auth.role !== role) {
      return res.status(403).json({ message: 'Forbidden' });
    }
    return next();
  };
};

```

Appendix D: BOOKING SCHEMA AND INVENTORY MODEL

```

// models/Booking.js — Stores bookings with nested inventory list
const mongoose = require('mongoose');

const InventoryItemSchema = new mongoose.Schema({
  item_name: { type: String, required: true }, // e.g. "King Bed", "Refrigerator"
  quantity: { type: Number, required: true, min: 1 },
  is_fragile: { type: Boolean, default: false },
  category: { type: String, enum: ['furniture', 'electronics', 'boxes', 'other'] },
}, { _id: false });

const BookingSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  mover: { type: mongoose.Schema.Types.ObjectId, ref: 'Mover' },
  source_address: { type: String, required: true },
  destination_address: { type: String, required: true },
  source_lat: Number,
  source_lng: Number,
  dest_lat: Number,
  dest_lng: Number,
  distance_km: { type: Number, required: true },
  move_date: { type: Date, required: true },
  inventory_list: { type: [InventoryItemSchema], validate: v => v.length > 0 },

```

```

selected_package: { type: String, enum: ['Basic', 'Standard', 'Premium'], required: true },
final_price: { type: Number, required: true, min: 0 },
status: {
  type: String,
  enum: ['PENDING', 'CONFIRMED', 'IN_TRANSIT', 'COMPLETED', 'CANCELLED'],
  default: 'PENDING'
},
created_at: { type: Date, default: Date.now }
});

```

```

BookingSchema.index({ user: 1, created_at: -1 });
BookingSchema.index({ mover: 1, status: 1 });

```

```

module.exports = mongoose.model('Booking', BookingSchema);

```

Appendix E: BOOKING DRAFT STATE MANAGEMENT

```

// store/bookingSlice.js — Redux Toolkit slice for the in-flight booking draft
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import api from '../api/client';

```

```

export const fetchPriceQuote = createAsyncThunk(
  'booking/fetchPriceQuote',
  async (payload, { rejectWithValue }) => {
    try {
      const { data } = await api.post('/pricing/quote', payload);
      return data; // { basic, standard, premium }
    } catch (err) {
      return rejectWithValue(err.response?.data || 'Network error');
    }
  }
);

```

```

export const createBooking = createAsyncThunk(
  'booking/create',
  async (payload, { rejectWithValue }) => {
    try {
      const { data } = await api.post('/bookings', payload);
      return data; // Booking document
    }
  }
);

```

```

    } catch (err) {
      return rejectWithValue(err.response?.data || 'Unable to create booking');
    }
  }
);

```

```

const initialState = {
  source: null,
  destination: null,
  distance_km: 0,
  inventory: [],          // [{ item_name, quantity, is_fragile, category }]
  move_date: null,
  quote: null,            // { basic, standard, premium }
  selected_package: null,
  recommended_package: null,
  booking: null,
  status: 'idle',         // idle | loading | success | error
  error: null
};

```

```

const bookingSlice = createSlice({
  name: 'booking',
  initialState,
  reducers: {
    setRoute(state, { payload }) {
      state.source = payload.source;
      state.destination = payload.destination;
      state.distance_km = payload.distance_km;
    },
    addItem(state, { payload }) {
      const idx = state.inventory.findIndex(i => i.item_name === payload.item_name);
      if (idx >= 0) state.inventory[idx].quantity += payload.quantity;
      else state.inventory.push(payload);
    },
    removeItem(state, { payload: name }) {
      state.inventory = state.inventory.filter(i => i.item_name !== name);
    },
    setMoveDate(state, { payload }) { state.move_date = payload; },
    selectPackage(state, { payload }) { state.selected_package = payload; },

```

```

    setRecommendation(state, { payload }) { state.recommended_package = payload; },
    resetDraft: () => initialState
  },
  extraReducers: (builder) => {
    builder
      .addCase(fetchPriceQuote.pending, (s) => { s.status = 'loading'; })
      .addCase(fetchPriceQuote.fulfilled, (s, a) => { s.status = 'success'; s.quote = a.payload;
    })
      .addCase(fetchPriceQuote.rejected, (s, a) => { s.status = 'error'; s.error = a.payload;
    })
      .addCase(createBooking.fulfilled, (s, a) => { s.booking = a.payload; });
  }
});

export const {
  setRoute, addItem, removeItem, setMoveDate,
  selectPackage, setRecommendation, resetDraft
} = bookingSlice.actions;

export default bookingSlice.reducer;

```

Appendix F: ADDRESS MAP SELECTION SCREEN

```

// screens/AddressMapScreen.js — Pick source & destination on an interactive map
import React, { useRef, useState } from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';
import MapView, { Marker, Polyline, PROVIDER_GOOGLE } from 'react-native-maps';
import * as Location from 'expo-location';
import { haversineKm } from '../utils/geo';

const INITIAL_REGION = {
  latitude: 28.6139, longitude: 77.2090, // Delhi centroid
  latitudeDelta: 0.15, longitudeDelta: 0.15
};

export default function AddressMapScreen({ navigation, route }) {
  const mapRef = useRef(null);
  const [source, setSource] = useState(null);
  const [destination, setDest] = useState(null);

```

```

const [mode, setMode] = useState('source'); // 'source' | 'destination'

const centreOnMe = async () => {
  const { status } = await Location.requestForegroundPermissionsAsync();
  if (status !== 'granted') return;
  const pos = await Location.getCurrentPositionAsync({});
  mapRef.current?.animateToRegion({
    latitude: pos.coords.latitude,
    longitude: pos.coords.longitude,
    latitudeDelta: 0.03, longitudeDelta: 0.03
  });
};

const onMapPress = (e) => {
  const coord = e.nativeEvent.coordinate;
  if (mode === 'source') { setSource(coord); setMode('destination'); }
  else { setDest(coord); }
};

const canContinue = source && destination;
const distance = canContinue
  ? haversineKm(source, destination).toFixed(1)
  : null;

return (
  <View style={styles.root}>
    <MapView
      ref={mapRef}
      provider={PROVIDER_GOOGLE}
      style={StyleSheet.absoluteFill}
      initialRegion={INITIAL_REGION}
      onPress={onMapPress}
    >
      {source && <Marker coordinate={source} pinColor="#16a34a" title="Pickup" />}
      {destination && <Marker coordinate={destination} pinColor="#dc2626"
title="Drop" />}
      {canContinue && (
        <Polyline
          coordinates={[source, destination]}

```



```

        strokeColor="#2563eb"
        strokeWidth={3}
      />
    )}
  </MapView>

  <View style={styles.hud}>
    <Text style={styles.hint}>
      {mode === 'source' ? 'Tap to set pickup point' : 'Now tap destination'}
    </Text>
    {distance && <Text style={styles.distance}>Route distance: {distance}
km</Text>}
    <TouchableOpacity style={styles.locBtn} onPress={centreOnMe}>
      <Text style={styles.locTxt}>Use my location</Text>
    </TouchableOpacity>
    <TouchableOpacity
      style={[styles.nextBtn, !canContinue && { opacity: 0.4 }]}
      disabled={!canContinue}
      onPress={() => navigation.navigate('Inventory', {
        source, destination, distance_km: Number(distance)
      })}
    >
      <Text style={styles.nextTxt}>Continue</Text>
    </TouchableOpacity>
  </View>
</View>
);
}

```

```

const styles = StyleSheet.create({
  root: { flex: 1 },
  hud: { position: 'absolute', bottom: 0, left: 0, right: 0,
    backgroundColor: '#fff', padding: 18, borderTopLeftRadius: 18,
    borderTopRightRadius: 18, elevation: 8 },
  hint: { fontSize: 15, marginBottom: 6 },
  distance: { fontSize: 16, fontWeight: '600', marginBottom: 10, color: '#111' },
  locBtn: { paddingVertical: 10, alignItems: 'center' },
  locTxt: { color: '#2563eb', fontWeight: '500' },
  nextBtn: { backgroundColor: '#2563eb', borderRadius: 10, paddingVertical: 14,

```

```

      alignItems: 'center', marginTop: 6 },
    nextTxt: { color: '#fff', fontWeight: '700', fontSize: 16 }
  });

```

Appendix G: MOBILE LOGIN SCREEN IMPLEMENTATION

// screens/LoginScreen.js — React Native login flow with AsyncStorage persistence

```

import React, { useState } from 'react';
import {
  View, Text, TextInput, TouchableOpacity,
  ActivityIndicator, StyleSheet, Alert
} from 'react-native';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { useDispatch } from 'react-redux';
import { setAuthUser } from '../store/authSlice';
import api from '../api/client';

export default function LoginScreen({ navigation }) {
  const dispatch = useDispatch();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [loading, setLoading] = useState(false);

  const handleLogin = async () => {
    if (!email || !password) {
      return Alert.alert('Missing', 'Enter both email and password');
    }
    try {
      setLoading(true);
      const { data } = await api.post('/auth/login', { email, password });
      await AsyncStorage.setItem('shift_token', data.token);
      dispatch(setAuthUser(data.user));
      navigation.replace('Home');
    } catch (err) {
      Alert.alert('Login failed', err.response?.data?.message || 'Try again');
    } finally {
      setLoading(false);
    }
  };

```

```

return (
  <View style={styles.container}>
    <Text style={styles.title}>Welcome back to Shifty</Text>

    <TextInput
      style={styles.input}
      placeholder="Email"
      autoCapitalize="none"
      keyboardType="email-address"
      value={email}
      onChangeText={setEmail}
    />
    <TextInput
      style={styles.input}
      placeholder="Password"
      secureTextEntry
      value={password}
      onChangeText={setPassword}
    />

    <TouchableOpacity style={styles.button} onPress={handleLogin} disabled=
{loading}>
      {loading
        ? <ActivityIndicator color="#fff" />
        : <Text style={styles.buttonText}>Sign in</Text>}
    </TouchableOpacity>

    <TouchableOpacity onPress={() => navigation.navigate('Signup')}>
      <Text style={styles.link}>New to Shifty? Create an account</Text>
    </TouchableOpacity>
  </View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: 24, justifyContent: 'center', backgroundColor: '#fff' },
  title: { fontSize: 26, fontWeight: '700', marginBottom: 24, color: '#111' },
  input: { borderWidth: 1, borderColor: '#ddd', borderRadius: 8,

```

```

        paddingHorizontal: 14, paddingVertical: 12, marginBottom: 14 },
    button: { backgroundColor: '#2563eb', borderRadius: 8,
        paddingVertical: 14, alignItems: 'center' },
    buttonText:{ color: '#fff', fontWeight: '600', fontSize: 16 },
    link: { marginTop: 18, textAlign: 'center', color: '#2563eb' }
});

```

Appendix H: RANDOM FOREST PACKAGE RECOMMENDER

ml/train_rf.py — Random Forest trainer for service package recommendation

```

import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import classification_report
from sklearn.preprocessing import LabelEncoder
import joblib

```

```

FEATURES = [
    'distance_km', 'total_items', 'fragile_ratio',
    'heavy_items', 'floor_source', 'floor_dest',
    'has_elevator', 'weekend_move'
]
TARGET = 'package'      # Basic | Standard | Premium

```

```

def load_training_data(path='data/bookings.csv'):
    df = pd.read_csv(path)
    df['fragile_ratio'] = df['fragile_count'] / df['total_items'].clip(lower=1)
    df['weekend_move'] = pd.to_datetime(df['move_date']).dt.weekday >= 5
    return df

```

```

def train():
    df = load_training_data()
    X, y = df[FEATURES], df[TARGET]

    le = LabelEncoder()
    y_enc = le.fit_transform(y)

    X_tr, X_te, y_tr, y_te = train_test_split(

```

```

X, y_enc, test_size=0.20, stratify=y_enc, random_state=42
)

clf = RandomForestClassifier(
    n_estimators=300,
    max_depth=14,
    min_samples_leaf=4,
    class_weight='balanced',
    n_jobs=-1,
    random_state=42
)
clf.fit(X_tr, y_tr)

cv_acc = cross_val_score(clf, X_tr, y_tr, cv=5).mean()
print(f'5-fold CV accuracy: {cv_acc:.3f}')

preds = clf.predict(X_te)
print(classification_report(y_te, preds, target_names=le.classes_))

joblib.dump({'model': clf, 'encoder': le, 'features': FEATURES},
            'artifacts/rf_package_v1.joblib')
print('Model saved -> artifacts/rf_package_v1.joblib')

if __name__ == '__main__':
    train()

```

Appendix I: XGBOOST DYNAMIC PRICING MODEL

```

# ml/train_xgb.py — XGBoost regressor for dynamic price prediction
import xgboost as xgb
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, r2_score
import joblib

FEATURES = [
    'distance_km', 'total_items', 'fragile_count',
    'heavy_items', 'package_enc', 'floor_source',

```

```
'floor_dest', 'has_elevator', 'day_of_week',  
'hour_of_day', 'fuel_price_index', 'demand_score'  
]
```

```
def build_dataset(path='data/completed_bookings.csv'):  
    df = pd.read_csv(path)  
    df['day_of_week'] = pd.to_datetime(df['move_date']).dt.weekday  
    df['hour_of_day'] = pd.to_datetime(df['move_date']).dt.hour  
    df['package_enc'] = df['package'].map({'Basic': 0, 'Standard': 1, 'Premium': 2})  
    return df
```

```
def train():  
    df = build_dataset()  
    X, y = df[FEATURES], df['final_price']  
  
    X_tr, X_te, y_tr, y_te = train_test_split(  
        X, y, test_size=0.15, random_state=7  
    )
```

```
    model = xgb.XGBRegressor(  
        n_estimators=600,  
        learning_rate=0.05,  
        max_depth=7,  
        min_child_weight=3,  
        subsample=0.85,  
        colsample_bytree=0.80,  
        objective='reg:squarederror',  
        tree_method='hist',  
        random_state=7  
    )  
    model.fit(X_tr, y_tr, eval_set=[(X_te, y_te)], verbose=False)
```

```
    preds = model.predict(X_te)  
    print(f'MAE : Rs.{mean_absolute_error(y_te, preds):.2f}')  
    print(f'R^2 : {r2_score(y_te, preds):.3f}')
```

```
joblib.dump({'model': model, 'features': FEATURES},  
            'artifacts/xgb_price_v3.joblib')
```

```
def predict_price(payload: dict) -> float:
    bundle = joblib.load('artifacts/xgb_price_v3.joblib')
    row = pd.DataFrame([payload])[bundle['features']]
    return float(bundle['model'].predict(row)[0])

if __name__ == '__main__':
    train()
```

Appendix J: LIVE TRACKING SOCKET GATEWAY

```
// realtime/trackingSocket.js — Socket.io handler for live mover -> customer GPS relay
const { Server } = require('socket.io');
const jwt = require('jsonwebtoken');
const Booking = require('../models/Booking');

function attachTrackingSocket(httpServer) {
    const io = new Server(httpServer, {
        cors: { origin: '*' },
        path: '/realtime'
    });

    // JWT handshake — reject unauthenticated sockets
    io.use((socket, next) => {
        const token = socket.handshake.auth?.token;
        if (!token) return next(new Error('UNAUTHORISED'));
        try {
            const payload = jwt.verify(token, process.env.JWT_SECRET);
            socket.user = { id: payload.sub, role: payload.role };
            return next();
        } catch { return next(new Error('BAD_TOKEN')); }
    });

    io.on('connection', (socket) => {
        console.log(`[socket] ${socket.user.role}:${socket.user.id} connected`);

        // Customer subscribes to a booking's live feed
        socket.on('track:subscribe', async ({ bookingId }) => {
            const booking = await Booking.findById(bookingId).lean();
            if (!booking) return socket.emit('track:error', 'Booking not found');
        });
    });
}
```

```

    if (String(booking.user) !== socket.user.id)
      return socket.emit('track:error', 'Forbidden');
    socket.join(`booking:${bookingId}`);
    socket.emit('track:subscribed', { bookingId });
  });

  // Mover publishes a GPS ping every 5 seconds
  socket.on('track:ping', async ({ bookingId, lat, lng, speed_kmh }) => {
    if (socket.user.role !== 'mover') return;
    const booking = await Booking.findById(bookingId).lean();
    if (!booking || String(booking.mover) !== socket.user.id) return;

    io.to(`booking:${bookingId}`).emit('track:location', {
      bookingId,
      lat, lng, speed_kmh,
      ts: Date.now()
    });
  });

  // Mover updates status e.g. PICKUP_DONE -> IN_TRANSIT -> DELIVERED
  socket.on('track:status', async ({ bookingId, status }) => {
    if (socket.user.role !== 'mover') return;
    await Booking.updateOne({ _id: bookingId, mover: socket.user.id }, { status });
    io.to(`booking:${bookingId}`).emit('track:status', { bookingId, status });
  });

  socket.on('disconnect', () => {
    console.log(`[socket] ${socket.user.role}:${socket.user.id} disconnected`);
  });
});

return io;
}

module.exports = attachTrackingSocket;

```


Appendix K: CONVERSATIONAL ASSISTANT ACTIONS

```
# chatbot/actions.py — Rasa custom actions for the Shifty assistant
from typing import Any, Dict, List, Text
import requests
from rasa_sdk import Action, Tracker
from rasa_sdk.events import SlotSet
from rasa_sdk.executor import CollectingDispatcher
```

```
API_BASE = 'http://localhost:4000/api'
```

```
class ActionQuoteEstimate(Action):
    """Returns a rough price estimate for a 3-tier package quote."""

    def name(self) -> Text:
        return 'action_quote_estimate'

    def run(self, dispatcher: CollectingDispatcher,
            tracker: Tracker,
            domain: Dict[Text, Any]) -> List[Dict[Text, Any]]:

        distance = tracker.get_slot('distance_km')
        items    = tracker.get_slot('item_count')
        fragile  = tracker.get_slot('fragile_count') or 0

        if not distance or not items:
            dispatcher.utter_message(
                text='I need the distance and item count to quote. '
                'Could you share your pickup and destination?'
            )
            return []

        try:
            resp = requests.post(f'{API_BASE}/pricing/quote', json={
                'distance_km': float(distance),
                'total_items': int(items),
                'fragile_count': int(fragile)
            }, timeout=5)
```

```

    q = resp.json()
except requests.RequestException:
    dispatcher.utter_message(text='Pricing service is unreachable, please retry.')
    return []

```

```

dispatcher.utter_message(
    text=(f"Here are your estimates:\n"
          f"- Basic    : Rs.{q['basic']}\n"
          f"- Standard : Rs.{q['standard']}\n"
          f"- Premium  : Rs.{q['premium']}\n"
          f"Recommended: {q['recommended']}")
)
return [SlotSet('last_quote', q)]

```

```

class ActionBookingStatus(Action):

```

```

    """Looks up status of the user's most recent booking."""

```

```

    def name(self) -> Text:
        return 'action_booking_status'

```

```

    def run(self, dispatcher, tracker, domain):
        token = tracker.get_slot('auth_token')
        if not token:
            dispatcher.utter_message(text='Please log in so I can look up your booking.')
            return []

```

```

        resp = requests.get(f'{API_BASE}/bookings/latest',
                             headers={'Authorization': f'Bearer {token}'})
        if resp.status_code != 200:
            dispatcher.utter_message(text='No active booking found.')
            return []

```

```

        b = resp.json()
        dispatcher.utter_message(
            text=(f"Booking #{b['_id'][-6:]} is currently *{b['status']}*.\n"
                  f"Mover: {b.get('mover_name', 'To be assigned')} "
                  f"Move date: {b['move_date'][:10]}")
        )

```

```
return []
```

```
class ActionCancelBooking(Action):
```

```
    def name(self): return 'action_cancel_booking'
```

```
    def run(self, dispatcher, tracker, domain):
```

```
        token = tracker.get_slot('auth_token')
```

```
        booking_id = tracker.get_slot('booking_id')
```

```
        if not (token and booking_id):
```

```
            dispatcher.utter_message(text='Which booking should I cancel?')
```

```
            return []
```

```
        resp = requests.patch(
```

```
            f'{API_BASE}/bookings/{booking_id}/cancel',
```

```
            headers={'Authorization': f'Bearer {token}'}
```

```
        )
```

```
        if resp.ok:
```

```
            dispatcher.utter_message(text='Your booking has been cancelled.')
```

```
        else:
```

```
            dispatcher.utter_message(text=resp.json().get('message', 'Could not cancel.'))
```

```
        return []
```

Appendix L: RAZORPAY PAYMENT CONTROLLER

```
// controllers/paymentController.js — Razorpay order creation + signature verification
```

```
const crypto = require('crypto');
```

```
const Razorpay = require('razorpay');
```

```
const Booking = require('../models/Booking');
```

```
const Payment = require('../models/Payment');
```

```
const rzp = new Razorpay({
```

```
    key_id: process.env.RAZORPAY_KEY_ID,
```

```
    key_secret: process.env.RAZORPAY_KEY_SECRET
```

```
});
```

```
// POST /api/payments/create-order
```

```
exports.createOrder = async (req, res) => {
```

```
    try {
```

```

const { bookingId } = req.body;
const booking = await Booking.findById(bookingId);
if (!booking) return res.status(404).json({ message: 'Booking not found' });
if (String(booking.user) !== req.auth.userId)
  return res.status(403).json({ message: 'Forbidden' });

const order = await rzp.orders.create({
  amount: Math.round(booking.final_price * 100), // paise
  currency: 'INR',
  receipt: `shifty_${booking._id}`,
  notes: { bookingId: String(booking._id) }
});

await Payment.create({
  booking: booking._id,
  user: booking.user,
  order_id: order.id,
  amount: booking.final_price,
  status: 'CREATED'
});

return res.json({
  orderId: order.id,
  amount: order.amount,
  currency: order.currency,
  key: process.env.RAZORPAY_KEY_ID
});
} catch (err) {
  console.error('createOrder', err);
  return res.status(500).json({ message: 'Payment initiation failed' });
}
};

// POST /api/payments/verify
exports.verifyPayment = async (req, res) => {
  const {
    razorpay_order_id,
    razorpay_payment_id,
    razorpay_signature,
  }

```

```

    bookingId
  } = req.body;

  const expected = crypto
    .createHmac('sha256', process.env.RAZORPAY_KEY_SECRET)
    .update(`${razorpay_order_id}|${razorpay_payment_id}`)
    .digest('hex');

  if (expected !== razorpay_signature) {
    await Payment.updateOne({ order_id: razorpay_order_id }, { status: 'FAILED' });
    return res.status(400).json({ message: 'Signature mismatch' });
  }

  await Payment.updateOne(
    { order_id: razorpay_order_id },
    { status: 'PAID', payment_id: razorpay_payment_id, paid_at: new Date() }
  );
  await Booking.updateOne(
    { _id: bookingId, user: req.auth.userId },
    { status: 'CONFIRMED' }
  );

  return res.json({ success: true, message: 'Payment verified' });
};

// Razorpay webhook -> reconcile async events
exports.webhook = async (req, res) => {
  const signature = req.headers['x-razorpay-signature'];
  const expected = crypto
    .createHmac('sha256', process.env.RAZORPAY_WEBHOOK_SECRET)
    .update(JSON.stringify(req.body))
    .digest('hex');

  if (expected !== signature) return res.status(400).send('bad signature');

  const event = req.body.event;
  if (event === 'payment.captured') {
    const p = req.body.payload.payment.entity;
    await Payment.updateOne({ order_id: p.order_id },

```

```
        { status: 'PAID', payment_id: p.id });  
    }  
    return res.json({ received: true });  
};
```

Appendix M: SNAPSHOTS OF SHIFTY RUNNING SYSTEM

The following snapshots are taken from the running Shifty prototype. They cover the landing page, the booking planner, the price prediction screen, the payment review screen, and the authentication flows.

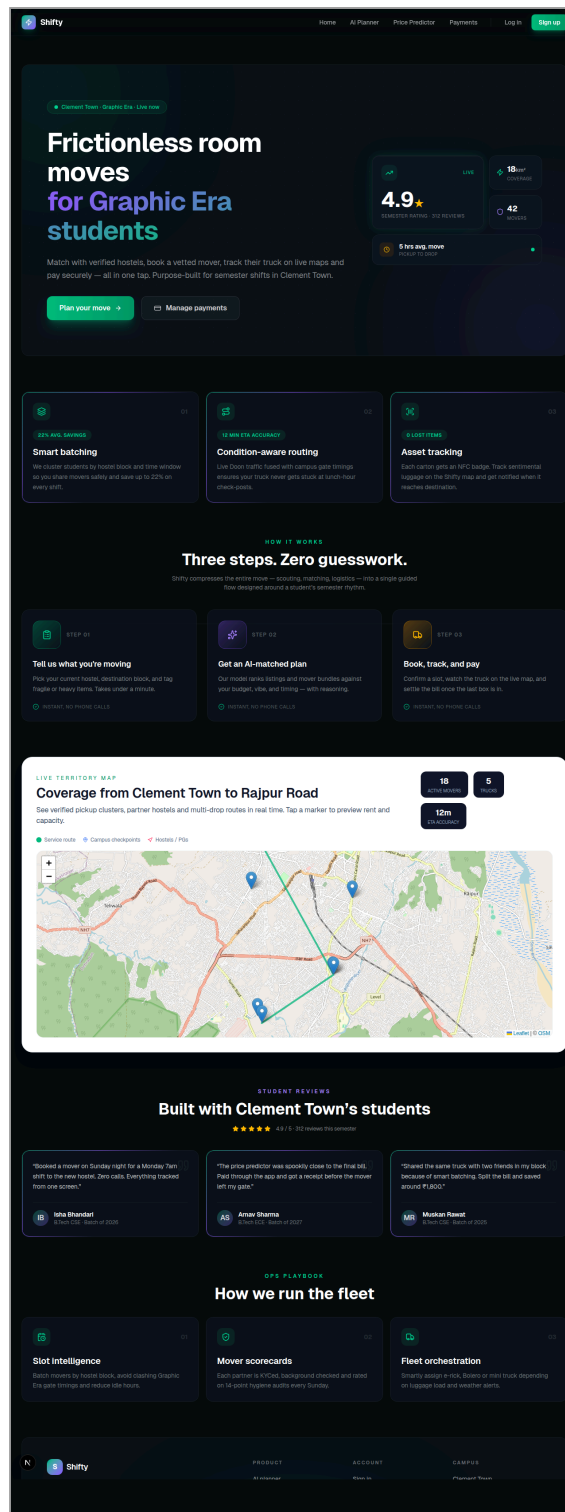


Figure 12: Landing Page of the Shifty Web Prototype

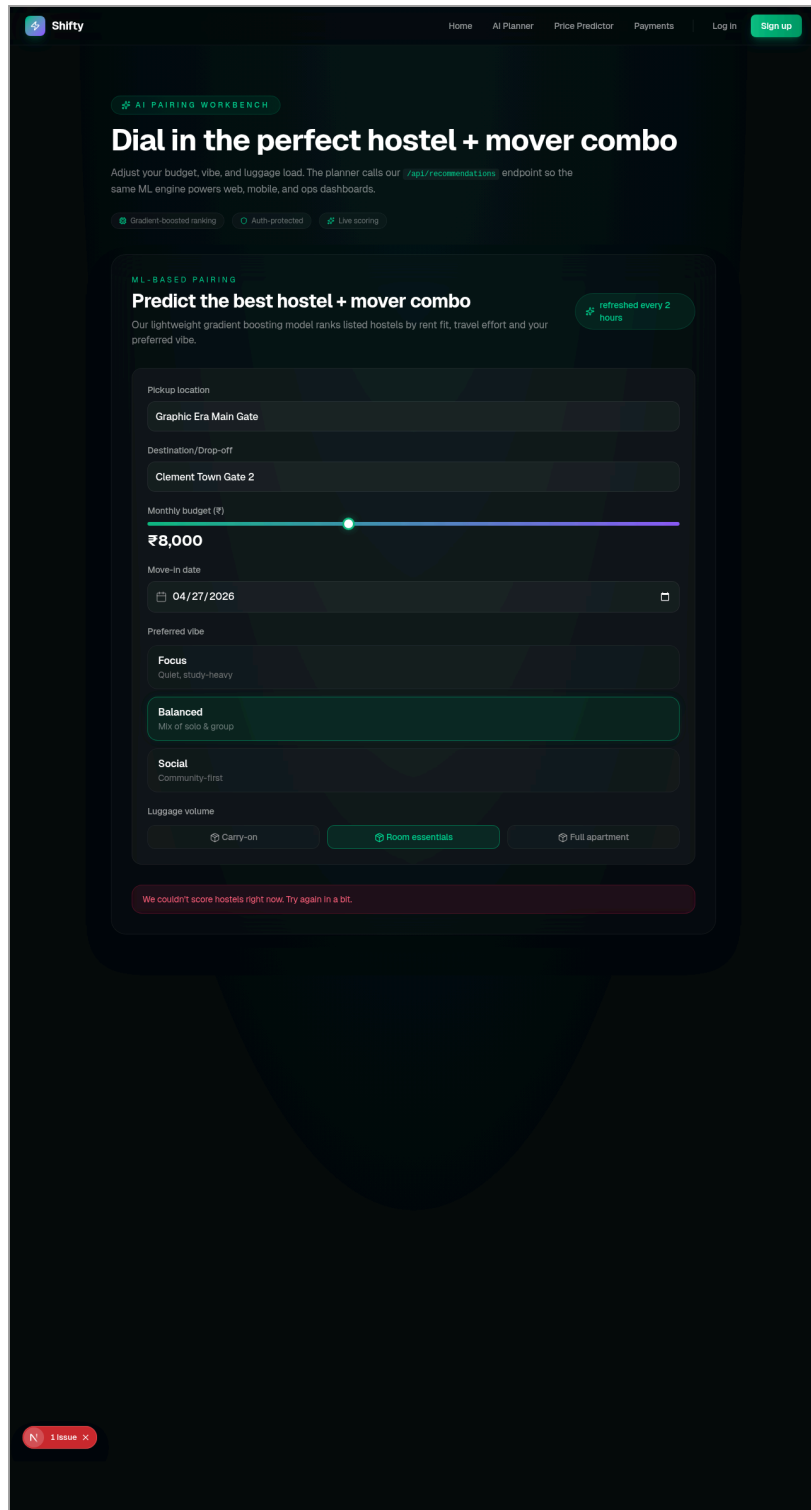


Figure 13: Booking Planner Capturing Itinerary and Inventory

Shifty

HomeAI PlannerPrice PredictorPaymentsLog InSign up

ML-POWERED PRICING ENGINE

Predict your move cost instantly

Our gradient-boosted regression model analyzes 7 input features — distance, luggage weight, floor level, time of day, urgency, fragile items, and rooms — to generate a real-time price prediction with confidence intervals and optimal package recommendations.

MODEL INPUTS

Distance5 km

Luggage weight60 kg

Floor levelFloor 2

Preferred time10 AM

Days until move3 days

Rooms1

Fragile Items

ML PREDICTED PRICE

97% confidence

High demand

₹2,862

₹2,819 – ₹2,905 range

Shifting to an afternoon slot (2–5 PM) could save ₹200–400.

DISTANCE₹943

LUGGAGE₹221

FLOOR+₹110

DEMAND1.22x

URGENCY+₹223

FRAGILE—

PRICE VS DISTANCE

PRICE VS TIME OF DAY

Choose your package

BEST VALUE

Economy

No frills, maximum savings

₹2,117

Standard

Best balance of price and care

₹2,862

Premium

White-glove service, zero stress

₹4,436

Figure 14: Price Prediction Screen Showing Model-Generated Quote

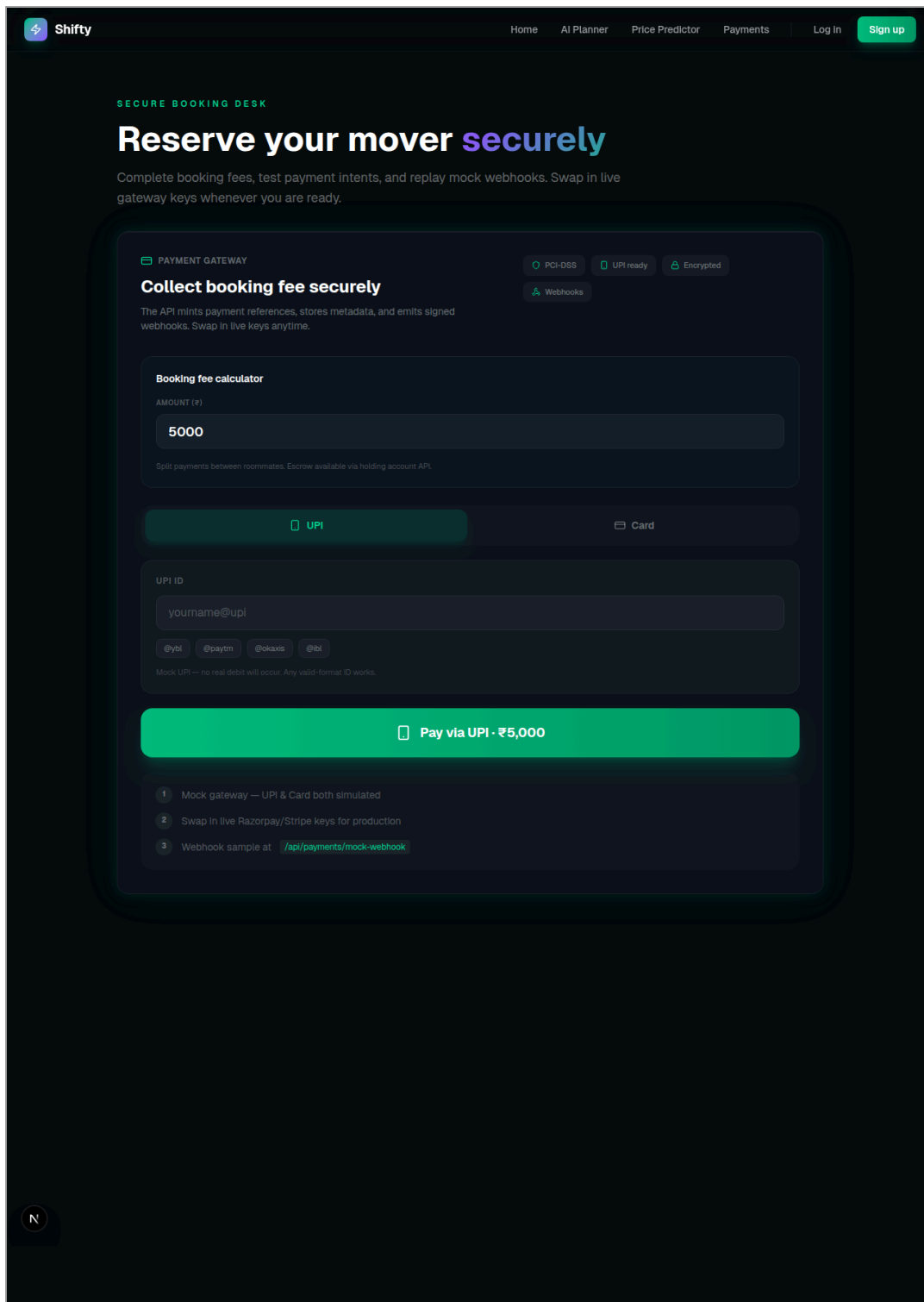


Figure 15: Payment Review Screen Prior to Gateway Hand-off

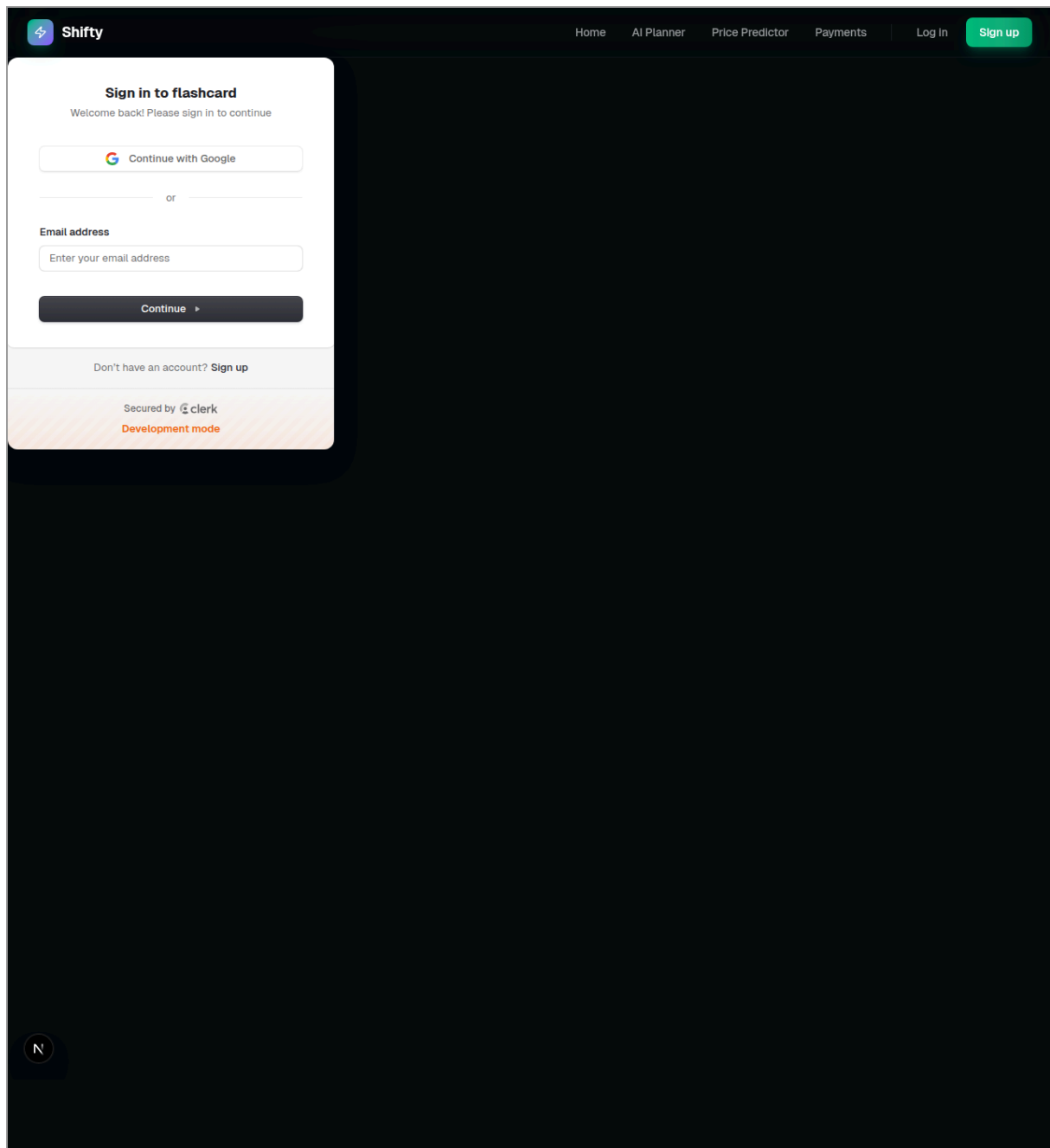


Figure 16: Sign-in Screen for Secure Access in Shifty System

 Shifty

[Home](#) [AI Planner](#) [Price Predictor](#) [Payments](#) [Log In](#) [Sign up](#)

CREATE ACCOUNT

Launch your Shifty signup in under a minute

Clerk handles OTP, passwordless and SSO for you. Once connected, students can verify their GEU IDs, reserve movers, and manage billing securely.

Create your account

Welcomel Please fill in the details to get started.



Continue with Google

or

First name

Optional

Last name

Optional

First name

Last name

Email address

Enter your email address

Password

Enter your password



Continue >

Already have an account? [Sign in](#)

Secured by  clerk

Development mode

N

Figure 17: Account Creation Screen with Inline Validation